Lecture Notes:
Class Imbalance

CIC1205 - Machine Learning

Eduardo Bezerra

ebezerra@cefet-rj.br

March 21, 2026

0 Contents

1 Introduction **2**
1.1 Motivation . . . 2
1.2 Running Example . . . 2

2 Why Imbalance Is a Problem **2**
2.1 The Accuracy Trap . . . 3
2.2 Why the Default Threshold Fails . . . 3
2.3 Training Bias Toward the Majority Class . . . 4
2.4 Misleading Model Comparisons . . . 4

3 Appropriate Metrics for Imbalanced Data **5**
3.1 Prefer Recall, Precision, and F_β . . . 5
3.2 The Decision Threshold α and the Precision–Recall Trade-off . . . 6
3.3 PR-AUC vs. ROC-AUC for Imbalanced Data . . . 6
3.4 Selecting the Optimal Threshold . . . 8

4 Data-Level Mitigation Strategies **10**
4.1 Random Undersampling . . . 10
4.2 Random Oversampling . . . 10
4.3 SMOTE: Synthetic Minority Oversampling Technique . . . 11
4.4 Combining Oversampling and Undersampling . . . 12
4.5 Informed Undersampling: Negative Sampling . . . 13

5 Algorithm-Level Mitigation Strategies **15**
5.1 Class-Weighted Loss . . . 15
5.2 Focal Loss . . . 15
5.3 Ensemble Methods: Balanced Bagging and EasyEnsemble . . . 16

6 The Threshold as a Hyperparameter **17**
6.1 Connecting Threshold Selection to Model Selection . . . 17
6.2 Why This View Matters . . . 17
6.3 Joint Selection of Model and Threshold . . . 18

7 Comparing Strategies: A Practical Overview **20**

8 Summary **21**

1 Introduction

1.1 Motivation

The evaluation framework developed in the *Model Evaluation* lecture notes assumes, largely implicitly, that the classes we want to predict are reasonably balanced: that the training data contains a representative mix of positive and negative examples, and that a model which correctly classifies the majority of examples is genuinely useful.

In practice, this assumption is frequently violated. Fraudulent bank transactions, faulty industrial components, rare diseases, and critical network intrusions are all examples of events that occur much less often than their absence. In such settings, the *minority class* is typically the one that matters most, yet it is the one for which the least evidence is available during training.

This combination (high stakes and low frequency) defines the **class imbalance problem**. These notes examine why imbalance causes standard machine learning pipelines to fail, how to recognize failure through appropriate metrics, and what strategies practitioners use to mitigate the problem at the data level, the algorithm level, and the decision level.

Prerequisites

These notes assume familiarity with the confusion matrix and the metrics derived from it (accuracy, precision, recall, F_1, ROC/AUC, PR curves), all covered in the *Model Evaluation* lecture notes. The concept of a decision threshold α is also used throughout.

1.2 Running Example

A fraud detection system monitors credit card transactions. Out of every 10,000 transactions processed in a day, roughly **10 are fraudulent** (0.1 % positive rate). A bank wants to flag fraudulent transactions automatically so that analysts can review them.

Class	**Count (per 10 000 transactions)**
Legitimate (negative, $y = 0$)	9,990
Fraudulent (positive, $y = 1$)	10
Total	10,000

This example will be used throughout to ground each concept concretely.

2 Why Imbalance Is a Problem

Class imbalance fundamentally alters the behavior of standard machine learning pipelines, often in ways that are not immediately visible through conventional evaluation practices. When one class overwhelmingly dominates the data, learning algorithms tend to prioritize performance on the majority class, since this alone is sufficient to achieve low overall error. As a result, models may appear highly accurate while failing almost completely

at detecting the minority class—the very outcomes that are typically of greatest interest. This section examines the mechanisms through which imbalance distorts both training and evaluation, highlighting why naive approaches break down and motivating the need for more appropriate metrics and modeling strategies.

2.1 The Accuracy Trap

The most immediate symptom of class imbalance is that **accuracy becomes a useless metric**. A classifier that always predicts "legitimate" achieves:

$$\text{Accuracy}_{\text{trivial}} = \frac{9{,}990}{10{,}000} = 99.9\,\%.$$

This sounds outstanding; the classifier detects *zero* fraudulent transactions. Its confusion matrix is:

Actual \ *Predicted*	Fraud ($\hat{y} = 1$)	Legit ($\hat{y} = 0$)	Total
Fraud ($y = 1$)	TP $= 0$	FN $= 10$	10
Legit ($y = 0$)	FP $= 0$	TN $= 9990$	9990
Total	0	10000	10000

Its recall for the positive class is Recall $= 0/(0 + 10) = 0$, and its F_1 score is 0. Accuracy hides this complete failure because the large TN count dominates the numerator. This is the **accuracy trap**.

2.2 Why the Default Threshold Fails

Even a well-trained classifier trained on imbalanced data tends to assign low probability scores to positive examples. The model implicitly reflects the base rate through training: its outputs will cluster near 0, and even genuinely fraudulent transactions may receive scores of, say, 0.10 to 0.30.

Applying the standard threshold $\alpha = 0.5$ in this setting will cause nearly all examples (including many true frauds) to be classified as negative. The result is high accuracy, near-zero recall, and a dangerously misleading performance report.

Connection to calibration

A perfectly *calibrated* model assigns predicted probabilities that match the true event frequencies in the data. Concretely, among all transactions assigned a score of, say, 0.001, approximately 0.1 % should be genuinely fraudulent. In a 0.1 % fraud dataset, most transactions will therefore receive scores close to zero; this is correct behaviour, not a flaw. The key point is that a model with low absolute scores can still be a strong *ranker*, which is why threshold-independent measures like AUC and PR-AUC are preferred for initial model comparison, while threshold tuning is a separate, subsequent step. Model calibration will be treated in detail in a later lecture.

2.3 Training Bias Toward the Majority Class

Regardless of the learning algorithm, a model trained on heavily imbalanced data is exposed to far more negative examples than positive ones. This creates a systematic pressure to favor the majority class, because doing so is already a low-error strategy on the training set. The precise mechanism through which this bias manifests depends on the algorithm family:

- **Decision trees** tend to produce splits that correctly classify the abundant negative class while placing all minority-class examples in a single impure leaf. The tree may never bother to subdivide that leaf further, since the impurity gain from doing so is small relative to the majority-class gains available elsewhere.
- **k-Nearest Neighbors** classifies a query point by majority vote among its k neighbors. In an imbalanced dataset, the neighborhood of a minority-class point tends to be dominated by majority-class examples, so the vote will go negative unless k is small or the minority class is locally dense.
- **Gradient-based models** (e.g., logistic regression, neural networks) minimize a loss function via iterative updates. On imbalanced data, each batch of gradient updates is dominated by negative examples. The gradient signal from the rare positive class is therefore small compared with that from the majority class, and the optimizer may converge to a solution that effectively ignores positives. This is sometimes called **majority-class bias in training**.

Figure 1 contrasts the class proportions in a balanced dataset with those of the fraud detection example. In the imbalanced case, the positive class is nearly invisible at 0.1 %, so any learning algorithm receives almost no corrective signal from minority-class examples during training.

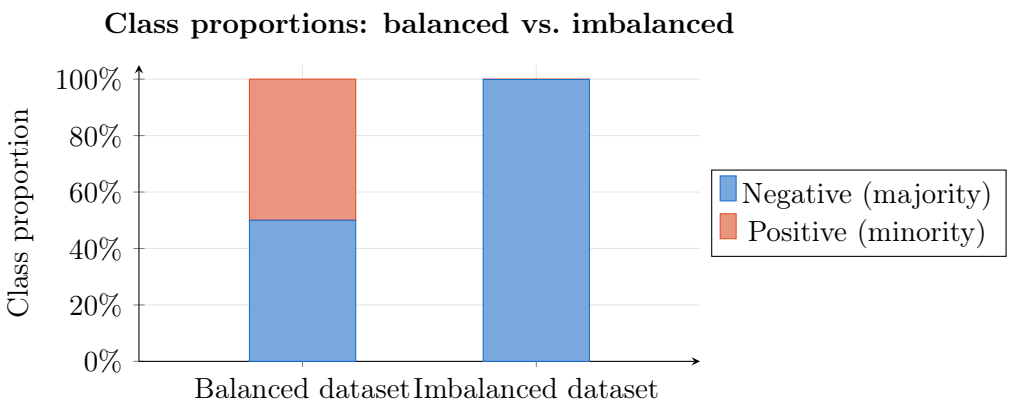

Figure 1: Class proportions for a balanced and an imbalanced dataset. In the fraud detection example, the positive class occupies 0.1 % of the data. Regardless of the learning algorithm, training is dominated by majority-class examples, creating systematic pressure to ignore the minority class.

2.4 Misleading Model Comparisons

Two models can have the same accuracy yet differ completely in their utility. Consider two classifiers evaluated on a held-out test set drawn from our fraud dataset:

Model	**Accuracy**	**Recall**	**Precision**	F_1
Always-negative trivial	99.9 %	0 %	N/A	0.000
Trained classifier	99.7 %	60 %	75 %	0.667

The trained classifier has *lower* accuracy than the trivial model yet is infinitely more useful. Reporting only accuracy would suggest the trivial model is better.

Entry Ticket 1

A medical screening test for a rare condition has a 1 % positive rate in the population. A model always predicts "healthy."

1. What is its accuracy on a test set of 1,000 patients?
2. What is its recall and F_1 score for the sick class?
3. Suppose a second model has accuracy 97 %, recall 80 %, and precision 50 %. Which model would you recommend to a hospital and why?

3 Appropriate Metrics for Imbalanced Data

In imbalanced classification problems, the choice of evaluation metric is not merely a technical detail but a central component of the modeling pipeline. Metrics such as accuracy, which aggregate performance across both classes, can obscure severe failures on the minority class and lead to misleading conclusions about model quality. To properly assess performance in this setting, we must adopt metrics that explicitly reflect the model's ability to detect rare but important events, as well as the reliability of its positive predictions. This section introduces such metrics, explains how they relate to the underlying decision threshold, and provides guidance on selecting evaluation criteria that align with the practical objectives of the application.

3.1 Prefer Recall, Precision, and F_β

Because accuracy is misleading on imbalanced data, the primary metrics should be those that focus on the minority class directly.

Metrics of choice for imbalanced problems

$$\text{Recall} = \frac{TP}{TP + FN} \qquad \text{(sensitivity to the minority class)}$$

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{(reliability of positive predictions)}$$

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \times \text{Recall}}{(\beta^2 \times \text{Precision}) + \text{Recall}} \qquad \text{(weighted harmonic mean)}$$

For $\beta = 1$ (equal weight): $F_1 = 2\,TP \,/\, (2\,TP + FP + FN)$.
For $\beta = 2$ (recall twice as important as precision): use F_2.
For $\beta = 0.5$ (precision twice as important as recall): use $F_{0.5}$.

In fraud detection, a false negative (missed fraud) is far more costly than a false positive (a legitimate transaction that is reviewed and cleared). This motivates $\beta > 1$, so that the metric penalizes missed frauds more heavily than false alarms.

3.2 The Decision Threshold α and the Precision–Recall Trade-off

Many classifiers do not output a hard label directly. Instead, they output a probability score $\hat{p} = \widehat{\Pr}(\text{class} = 1 \mid \mathbf{x})$. A **decision threshold** α converts this score into a prediction:

$$\hat{y} = \begin{cases} 1 & \text{if } \hat{p} \geq \alpha, \\ 0 & \text{otherwise.} \end{cases}$$

The default choice $\alpha = 0.5$ is rarely appropriate for imbalanced data. The threshold is a lever that controls the fundamental trade-off between precision and recall:

- **Raising** α: the model predicts positive only when it is very confident. Precision increases (reliable positive predictions), recall decreases (more true positives are missed).
- **Lowering** α: the model predicts positive more liberally. Recall increases (fewer true positives are missed), precision decreases (more false positives are raised).

Threshold sweep example (fraud detection). Suppose a trained fraud classifier is evaluated at three thresholds on a **validation set** with 100 fraudulent and 9,900 legitimate transactions. (Recall that the threshold is being *chosen* here, so it must be selected on validation data — not on the test set, which is reserved for the single final evaluation of the deployed model.)

α	TP	FP	FN	TN	Precision	Recall
0.50 (default)	30	5	70	9895	0.857	0.300
0.20 (aggressive)	70	80	30	9820	0.467	0.700
0.10 (very low)	88	300	12	9600	0.227	0.880

At the default threshold, only 30 % of frauds are caught. Lowering to 0.10 catches 88 % of frauds, at the cost of flagging 300 legitimate transactions for review. The right threshold depends entirely on the **cost structure** of the application. Once selected on the validation set, the chosen α is fixed; the test set is then evaluated *once* at that threshold to obtain an unbiased performance estimate. The correct procedure for jointly selecting the model and the threshold within cross-validation is detailed in Section 6.

Figure 2 shows the full PR curve for this classifier, constructed from validation-set scores, with the three operating points marked. The curve makes the trade-off visible across all thresholds at once: moving right (lower α) recovers more frauds but at increasingly lower precision.

3.3 PR-AUC vs. ROC-AUC for Imbalanced Data

Both the PR curve and the ROC curve summarize threshold behavior, but they respond very differently to class imbalance.

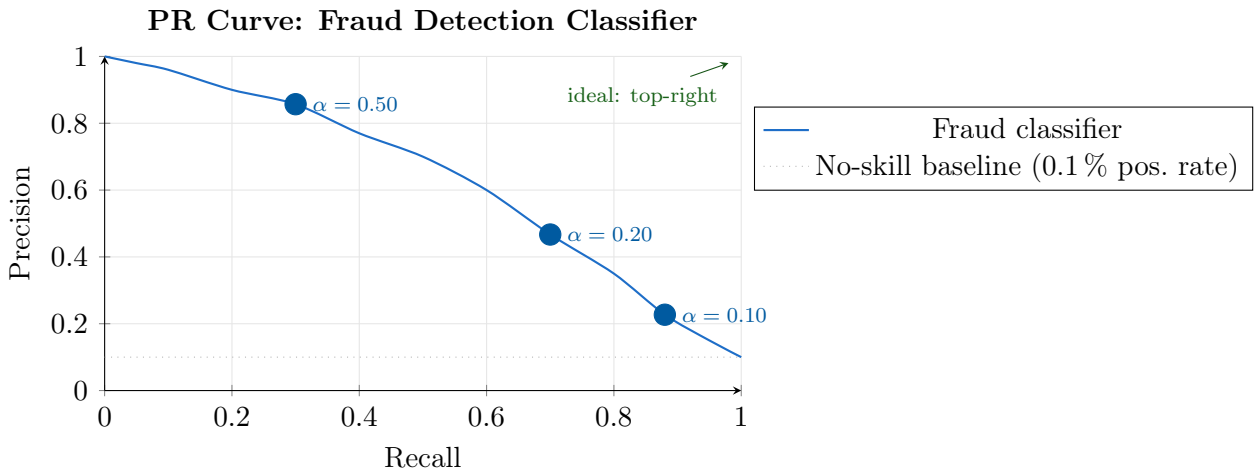

Figure 2: PR curve for a fraud detection classifier, constructed from validation-set scores. Three operating points corresponding to different thresholds are marked. The no-skill baseline is at 0.1 % (the fraction of fraudulent transactions). Moving left to right along the curve corresponds to lowering α: recall increases but precision decreases sharply. Because the threshold is being *selected* here, the curve must be built on validation data; the test set is used only once, at the chosen operating point.

ROC curve and AUC

The ROC curve plots TPR (= Recall) against FPR = $FP/(FP+TN)$ as α varies. Its area under the curve (AUC-ROC) equals the probability that a randomly chosen positive example is scored higher than a randomly chosen negative example (with ties handled appropriately).

Why ROC-AUC can mislead on imbalanced data. The FPR involves TN: FPR = $FP/(FP+TN)$. When the negative class is enormous (e.g., 9,990 legitimate transactions), TN is huge and FPR stays small even when many false positives are produced. For example, 300 false positives out of 9,900 negatives gives FPR = 300/9,900 $\approx$ 3 %, a number that looks very benign on the ROC plot, even though 300 unnecessary fraud flags are a real operational burden.

The PR curve avoids this problem by never involving TN. Precision = $TP/(TP+FP)$ is directly penalized by false positives, regardless of how many true negatives exist.

When to use PR-AUC vs. AUC-ROC

- Use **PR-AUC** when positive examples are rare and false positives are operationally costly. PR-AUC gives an honest picture of how well the model performs on the minority class.
- Use **AUC-ROC** when classes are relatively balanced, or when you want a threshold-independent summary of the model's overall ranking ability.
- For highly imbalanced problems, report *both*: PR-AUC as the primary evaluation criterion and AUC-ROC as a complementary ranking measure.

Entry Ticket 2

A classifier for rare network intrusions (0.5 % positive rate) achieves AUC-ROC = 0.97 on a test set. A colleague claims this is an excellent model.

1. Explain why a high AUC-ROC does not guarantee that the model is useful in this imbalanced setting. Which cell of the confusion matrix is responsible for inflating AUC-ROC?
2. What additional metric would you request before accepting the colleague's claim?
3. Suppose the classifier flags 5,000 alerts per day on a network that has 100 true intrusions. What is its precision? Why does this matter operationally?

3.4 Selecting the Optimal Threshold

There is no universally correct threshold. The right choice depends on the cost structure of the problem and, when available, operational constraints.

Strategy 1: Maximize a Target Metric

Sweep α over a grid of values on a validation set and choose the value that maximizes the metric most aligned with the application objective:

- F_β **score**: maximizes the harmonic mean of precision and recall weighted by β. Choose $\beta > 1$ to prioritize recall (minimize FN), or $\beta < 1$ to prioritize precision (minimize FP).
- **Youden's J statistic**: $J = \text{TPR} - \text{FPR}$. Maximizing J balances sensitivity and specificity without weighting either class. Read directly from the ROC curve as the threshold that maximizes the vertical distance from the diagonal.

Strategy 2: Cost-Sensitive Threshold Selection

When the cost of each error type can be quantified, we compute the **expected cost** at each threshold and choose its minimizer.

Let c_{FN} be the cost of a false negative and c_{FP} the cost of a false positive. At threshold α, the expected cost per transaction is:

$$\mathcal{C}(\alpha) = c_{\text{FN}} \cdot \text{FN}(\alpha) + c_{\text{FP}} \cdot \text{FP}(\alpha).$$

Numerical example (fraud detection). Suppose $c_{\text{FN}} = \$500$ (average loss from undetected fraud) and $c_{\text{FP}} = \$5$ (cost of reviewing a legitimate transaction). Using the validation-set threshold sweep from Section 3:

α	FN	FP	$\mathcal{C}(\alpha)$	
0.50	70	5	$70 \times \$500 + 5 \times \$5 = \$35{,}025$	
0.20	30	80	$30 \times \$500 + 80 \times \$5 = \$15{,}400$	
0.10	12	300	$12 \times \$500 + 300 \times \$5 = \$7{,}500$	$\leftarrow$ optimal

The cost is lowest at $\alpha = 0.10$. This is driven by the large cost ratio $c_{\text{FN}}/c_{\text{FP}} = 100$: each missed fraud is 100 times more expensive than a false alarm, so it pays to lower the threshold aggressively. Note that with different cost assumptions the optimal threshold would shift; always re-derive this analysis for the specific application.

Strategy 3: Operational Constraints

In deployed systems, the threshold is often set to satisfy a business rule:

- *"Flag at most N transactions per day for review."* This translates to a maximum FP budget; read the corresponding threshold from the validation-set PR curve.
- *"Catch at least 90 % of frauds."* This translates to a minimum recall target; find the highest α that achieves Recall ≥ 0.90 on the validation set.

Figure 3 plots precision, recall, and F_1 as explicit functions of α, computed on the validation set. This view complements the PR curve: instead of one metric against the other, each metric is shown as a function of the decision variable, making it straightforward to read off the threshold that satisfies any single-metric constraint.

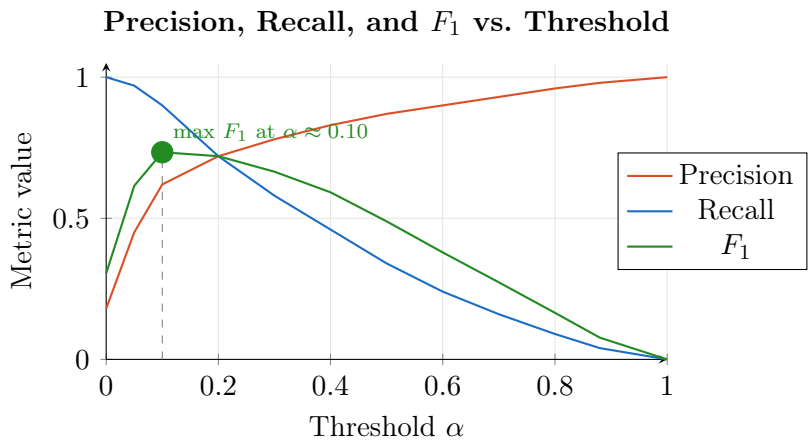

Figure 3: Precision, recall, and F_1 as functions of the threshold α for the fraud classifier, evaluated on the validation set. Lower thresholds increase recall but reduce precision, whereas higher thresholds do the opposite. In this example, the best balance between the two is achieved near $\alpha = 0.10$, where the F_1 score reaches its maximum. All threshold comparisons must be performed on validation data; the test set is reserved for a single final evaluation at the chosen $\hat{\alpha}$.

Entry Ticket 3

A classifier for loan default (5 % positive rate) outputs a probability score. On a validation set of 200 loans (10 defaults, 190 non-defaults), you record the following at three thresholds:

α	TP	FP	FN	TN
0.50	4	2	6	188
0.30	7	8	3	182
0.15	9	25	1	165

1. Compute precision, recall, and F_2 score at each threshold.

2. A missed default costs the bank \$2,000; a false positive costs \$30 (call to customer service). Compute $\mathcal{C}(\alpha)$ at each threshold and identify the cost-optimal choice.
3. Regulations require the bank to catch at least 80 % of defaults. Which threshold satisfies this constraint while minimizing false positives?

4 Data-Level Mitigation Strategies

Data-level strategies address class imbalance by modifying the training set before fitting the model. The goal is to expose the learning algorithm to a more balanced view of the two classes.

4.1 Random Undersampling

Random undersampling removes examples from the majority class at random until the desired class ratio is achieved.

Procedure. Given N_- negative and N_+ positive examples ($N_- \gg N_+$), randomly select N_+ (or $r \cdot N_+$ for some ratio r) negative examples and discard the rest.

Advantages.

- Extremely fast and simple to implement.
- Reduces training time for large datasets.

Disadvantages.

- Discards potentially informative majority-class examples.
- With aggressive undersampling, the model may lose important decision boundary information from the majority class.

Numerical illustration. From our fraud dataset (9,990 legitimate, 10 fraudulent):

- 1:1 ratio: keep 10 legitimate examples (discard 9,980). Dataset size: 20.
- 10:1 ratio: keep 100 legitimate examples (discard 9,890). Dataset size: 110.
- 100:1 ratio (mild): keep 1,000 legitimate examples (discard 8,990). Dataset size: 1,010.

4.2 Random Oversampling

Random oversampling duplicates minority-class examples at random until the desired class ratio is achieved.

Procedure. Sample from the N_+ positive examples with replacement until the positive class has grown to the desired size.

Advantages.

- No information is discarded from the majority class.
- Simple to implement.

Disadvantages.

- Exact duplication can cause the model to overfit to the duplicated examples.
- Increases training time proportionally to the oversampling factor.

4.3 SMOTE: Synthetic Minority Oversampling Technique

SMOTE [Chawla et al., 2002] generates *synthetic* minority examples by interpolating between existing ones, rather than simply duplicating them.

SMOTE Algorithm (one synthetic example)

For each positive example $\mathbf{x}^+$:

1. Find the k nearest neighbors of $\mathbf{x}^+$ among the positive class (typically $k = 5$).
2. Choose one neighbor $\mathbf{x}^+_{\text{nn}}$ at random.
3. Generate a synthetic example:

$$\mathbf{x}_{\text{syn}} = \mathbf{x}^+ + \lambda\,(\mathbf{x}^+_{\text{nn}} - \mathbf{x}^+), \quad \lambda \sim \text{Uniform}(0, 1).$$

The synthetic example lies on the line segment between $\mathbf{x}^+$ and its neighbor, at a random position.

Figure 4 illustrates one step of this procedure. A real minority example $\mathbf{x}^+$ is connected to one of its nearest minority neighbors; the synthetic point is placed at a random position along that segment, encouraging it to lie within the minority-class region of the feature space (though this is not guaranteed when classes overlap).

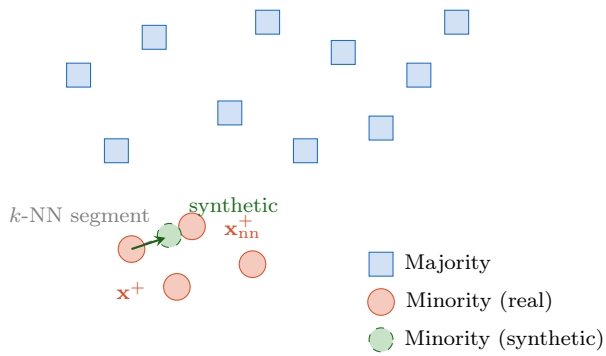

Figure 4: SMOTE generates synthetic minority examples (green dashed circles) by interpolating between a real minority example $\mathbf{x}^+$ and one of its k nearest minority neighbors. The synthetic point lies on the line segment connecting the two, at a random position $\lambda \in (0, 1)$. In regions where the minority and majority classes overlap, synthetic points may land in ambiguous territory; extensions such as Borderline-SMOTE address this by restricting generation to the decision-boundary region.

Advantages over random oversampling.

- Reduces the risk of overfitting to duplicated points because each synthetic example is unique.
- Encourages the model to learn smoother decision boundaries in the minority region.

Limitations.

- SMOTE can generate synthetic examples in regions where the minority class overlaps with the majority class, potentially introducing noise.
- Purely feature-space interpolation may not make sense for high-dimensional or categorical features.
- Extensions such as **Borderline-SMOTE** (generates only near the decision boundary) and **ADASYN** (generates more in harder regions) address some of these issues.

Practical usage (scikit-learn).

```
from imblearn.over_sampling import SMOTE
sm = SMOTE(sampling_strategy='minority', k_neighbors=5, random_state=42)
X_resampled, y_resampled = sm.fit_resample(X_train, y_train)
```

This code snippet illustrates how SMOTE is applied using the `imbalanced-learn` library. The `SMOTE` object is instantiated with `sampling_strategy='minority'`, which indicates that only the minority class should be oversampled until a desired balance is reached, and `k_neighbors=5`, specifying how many nearest neighbors are used to generate each synthetic example. The method `fit_resample` is then called on the training data `(X_train, y_train)`, producing a new dataset `(X_resampled, y_resampled)` in which additional synthetic minority examples have been created. Importantly, this operation must be performed only on the training data (and inside each cross-validation fold, when applicable) to avoid data leakage into validation or test sets.

Critical warning: apply resampling inside cross-validation

SMOTE and other resampling methods must be applied **only to the training fold**, never to the validation or test fold. If SMOTE is applied before the train/validation split, synthetic examples from the minority class may appear in the validation fold, creating data leakage and inflated metrics.
The correct approach in scikit-learn is to use a `Pipeline` that includes the resampler as a step; the pipeline ensures that resampling happens inside each cross-validation fold, touching only the training data.

4.4 Combining Oversampling and Undersampling

In practice, a combination of SMOTE (or random oversampling) with random undersampling often outperforms either alone. For example:

- Apply SMOTE to bring the minority class from 0.1 % to 5 %.
- Apply random undersampling to bring the majority class down to 10 %.

- The result (5 % positive rate) is still imbalanced but tractable.

This avoids the extreme expansion of the training set that pure oversampling would require (growing from 10 to 9,990 synthetic frauds) while retaining more majority-class information than pure undersampling.

Entry Ticket 4

A training set has 5,000 examples: 4,950 negative and 50 positive.

1. If you apply random oversampling to reach a 1:1 ratio, how many synthetic positive examples will be generated? What is the risk of this approach?
2. If you apply SMOTE with $k = 5$ to generate 950 synthetic positives, explain step by step how one synthetic example is constructed.
3. Why must SMOTE be applied inside the training fold of cross-validation, not before the split?

4.5 Informed Undersampling: Negative Sampling

Random undersampling discards majority-class examples uniformly at random, without regard to how informative each example is. **Negative sampling** is a family of undersampling strategies that replaces this random selection with a *criterion-driven* one: only the most informative negative examples are retained, while uninformative ones are discarded.

The underlying intuition is that not all negative examples contribute equally to learning. Easy negatives (i.e., examples that are trivially far from the decision boundary and correctly classified with high confidence) add little to the model's knowledge of where the boundary lies. Retaining them wastes representational capacity and reinforces majority-class bias. Hard negatives, by contrast, lie close to the decision boundary and are more easily confused with positives; they carry the most information about the structure of the problem.

Figure 5 illustrates the core intuition. In the left panel, the majority class occupies the full negative region: some examples lie close to the decision boundary (hard negatives) while many others are scattered far from it (easy negatives). After negative sampling, shown in the right panel, the easy negatives are discarded and only the hard negatives are retained. The resulting training set is smaller, but the proportion of examples that carry information about the decision boundary is much higher. The positive examples are left untouched throughout; the intervention is entirely on the majority side.

Two common criteria for selecting which negatives to keep are:

- **Proximity to the minority class**: retain the negative examples that are nearest (in feature space) to positive examples. Methods such as **NearMiss** implement variants of this idea, differing in whether they select negatives close to the nearest positive, the farthest positive, or the centroid of the positive class.
- **Classification difficulty**: train a preliminary classifier and retain the negative examples on which it is least confident (predicted probability closest to 0.5). This is sometimes called **hard negative mining** and is particularly common in deep learning pipelines for object detection and metric learning.

A related idea is to use a **pilot model** — a simple classifier trained first on the full

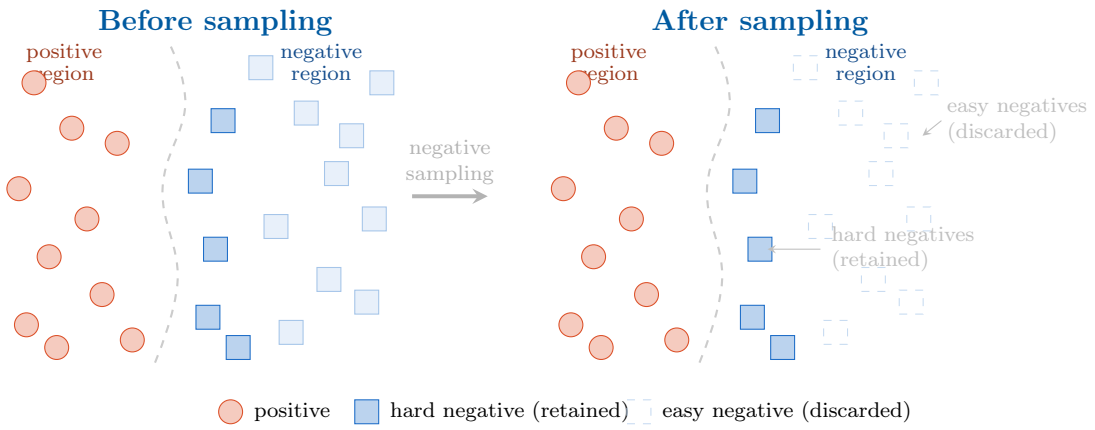

Figure 5: Intuition behind negative sampling. In the left panel, the training set contains positive examples (orange circles), hard negatives close to the decision boundary (solid blue squares), and easy negatives far from it (faint blue squares). After negative sampling (right panel), easy negatives are discarded; only the hard negatives near the boundary are retained. The retained set is smaller but more informative: it concentrates the learning signal on the region where the two classes are difficult to separate.

imbalanced data — to guide the undersampling step. Rather than defining hardness geometrically (as NearMiss does), the pilot model assigns a difficulty score to each negative example based on its predicted probability: negatives that the pilot model already classifies correctly with high confidence are deemed uninformative and discarded, while those near the decision boundary are retained. The main model is then trained on this filtered set. This two-stage approach appears in several methods: **BalanceCascade** removes easy negatives iteratively after each round of an ensemble; **Tomek Links** and **Edited Nearest Neighbours (ENN)** use a local neighbourhood model to identify and remove negatives that straddle the class boundary. In all cases the pilot model serves the same role: replacing blind random removal with an informed selection that preserves the examples most useful for learning the decision boundary.

Advantages over random undersampling.

- Retains the majority-class examples most relevant to the decision boundary, rather than discarding them at random.
- Can produce a more informative training set with the same or smaller size.

Disadvantages.

- NearMiss-style methods are sensitive to noise: a noisy positive example will attract nearby negatives that are not genuinely hard, polluting the retained set.
- Pilot-model approaches require an additional training step and inherit any biases of the preliminary classifier.
- Like all undersampling methods, informed undersampling discards data and may perform poorly when the dataset is already small.

5 Algorithm-Level Mitigation Strategies

Algorithm-level strategies modify the learning process itself to make the model more sensitive to the minority class, without altering the training set.

5.1 Class-Weighted Loss

The most direct algorithm-level fix is to penalize misclassifications of the minority class more heavily by assigning a **higher loss weight** to positive examples.

For binary cross-entropy, the standard unweighted loss per example is:

$$\ell(y, \hat{p}) = -\big[y \log \hat{p} + (1 - y) \log(1 - \hat{p})\big].$$

The **class-weighted** variant assigns weight w_+ to positive examples and w_- to negative examples:

$$\ell_w(y, \hat{p}) = -\big[w_+ \cdot y \log \hat{p} + w_- \cdot (1 - y) \log(1 - \hat{p})\big].$$

A common choice is the **inverse frequency weighting**:

$$w_+ = \frac{N}{2\,N_+}, \qquad w_- = \frac{N}{2\,N_-},$$

where $N = N_+ + N_-$ is the total training size. This makes the effective contribution of each class equal regardless of its frequency.

Numerical example. With $N_+ = 10$, $N_- = 9{,}990$, $N = 10{,}000$:

$$w_+ = \frac{10{,}000}{2 \times 10} = 500, \qquad w_- = \frac{10{,}000}{2 \times 9{,}990} \approx 0.5.$$

Each fraudulent transaction contributes 1,000 times more to the loss than each legitimate transaction, compensating for the 1,000-fold class imbalance.

In practice (scikit-learn and Keras).

```
# scikit-learn (e.g., LogisticRegression, RandomForestClassifier)
clf = LogisticRegression(class_weight='balanced')

# Keras / TensorFlow
class_weight = {0: 0.5, 1: 500}
model.fit(X_train, y_train, class_weight=class_weight, ...)
```

5.2 Focal Loss

Focal loss [Lin et al., 2017], originally proposed for object detection, addresses a subtler problem: on imbalanced datasets, a well-trained model assigns very high confidence to easy negative examples. These easy examples dominate the loss even though they carry little information.

Focal loss down-weights easy examples dynamically by multiplying the standard cross-entropy by a modulating factor $(1-\hat{p}_t)^\gamma$, where $\hat{p}_t$ is the predicted probability of the true class:

$$\ell_{\text{focal}}(y, \hat{p}) = -(1-\hat{p}_t)^\gamma \log(\hat{p}_t), \quad \hat{p}_t = \begin{cases} \hat{p} & \text{if } y = 1, \\ 1-\hat{p} & \text{if } y = 0. \end{cases}$$

When $\hat{p}_t \approx 1$ (easy example), $(1-\hat{p}_t)^\gamma \approx 0$ and the loss is suppressed. When $\hat{p}_t \approx 0$ (hard example), $(1-\hat{p}_t)^\gamma \approx 1$ and the loss is unchanged. The hyperparameter $\gamma \geq 0$ controls the rate of down-weighting; $\gamma = 0$ recovers standard cross-entropy.

Figure 6 compares the loss curves for several values of γ. When the model is already confident ($\hat{p}_t$ near 1), focal loss suppresses the gradient contribution of that example; when the model is uncertain ($\hat{p}_t$ near 0), the loss is essentially unchanged relative to standard cross-entropy.

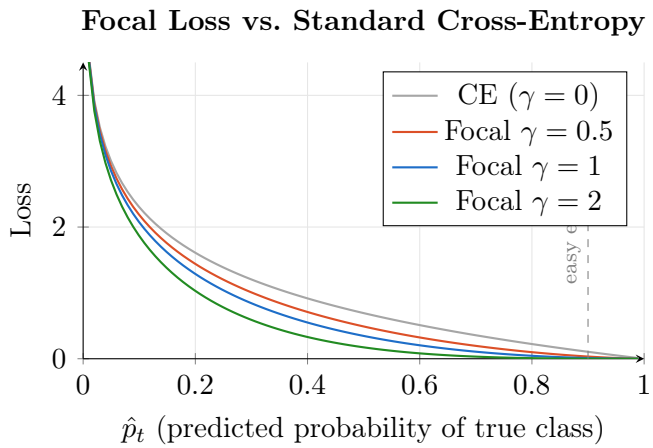

Figure 6: Focal loss for different values of γ. When a model assigns high probability to the true class ($\hat{p}_t$ near 1), easy examples are down-weighted regardless of their class. The larger γ is, the more aggressively easy examples are suppressed, which in practice focuses learning on hard examples that often (but not exclusively) belong to the minority class.

5.3 Ensemble Methods: Balanced Bagging and EasyEnsemble

Ensemble methods combine multiple classifiers trained on different data subsets. Two variants are specifically designed for imbalanced learning:

Balanced Bagging. Each bootstrap sample used to train a base classifier is resampled to achieve a target class ratio. The final prediction is the average or vote across all base classifiers. Because each base classifier sees a balanced sample, the ensemble is more sensitive to the minority class than a single classifier trained on the full imbalanced set.

EasyEnsemble [Liu et al., 2009]. The majority class is partitioned into several disjoint subsets, each of which is combined with all minority-class examples to form a balanced training set for one base classifier. The results are aggregated across all base classifiers. This ensures that all majority-class examples are used across the ensemble while each individual classifier sees a balanced sample.

Entry Ticket 5

A logistic regression model is trained on a dataset with $N_+ = 50$ positives and $N_- = 4{,}950$ negatives ($N = 5{,}000$).

1. Compute the inverse-frequency class weights w_+ and w_-.
2. Suppose a fraudulent transaction (true label 1) is assigned $\hat{p} = 0.3$ by the model. Compute the standard cross-entropy loss and the focal loss with $\gamma = 2$. How does the ratio $\ell_{\text{focal}}/\ell_{\text{CE}}$ compare?
3. A legitimate transaction (true label 0) is assigned $\hat{p} = 0.05$. Compute the same ratio. What does this reveal about which type of example focal loss down-weights more?

6 The Threshold as a Hyperparameter

6.1 Connecting Threshold Selection to Model Selection

Throughout Section 3 we treated the decision threshold α as a tunable quantity whose value is chosen after training to optimize a performance objective. It is worth making this connection explicit: **the threshold α is a hyperparameter of the deployed system**, and selecting it is an instance of the same *model selection* activity covered in the companion lecture notes.

Recall the key distinction from those notes:

- **Model evaluation** asks how well a fixed, fully specified model performs on unseen data.
- **Model selection** asks which among several candidate configurations should be deployed, where the choice is guided by performance on a *validation* set.

Choosing α fits squarely in the second category. We evaluate multiple threshold values on held-out data and select the one that optimizes the target criterion. The selected α, together with the fitted model weights $\hat{\theta}$, constitutes the final deployed system.

6.2 Why This View Matters

Treating α as just another hyperparameter has several concrete implications:

Data leakage applies. If we select α by sweeping over the *test* set, the test-set performance at that threshold is no longer an unbiased estimate of generalization: we have effectively used test labels to make a modeling decision. The correct procedure is to select α on a *validation* set (or via inner cross-validation folds) and report performance at the chosen threshold on the test set only once.

The threshold is part of the model description. When reporting results, the threshold used must always be stated alongside the metric values. A precision of 0.90 at $\alpha = 0.80$ and a precision of 0.45 at $\alpha = 0.20$ describe very different operating regimes of the same model; without α, the numbers are meaningless.

Multiple thresholds can be compared like models. If the target metric is F_β, one can frame threshold selection as: "among all candidate thresholds $\{\alpha_1, \alpha_2, \ldots\}$, which one yields the highest F_β on the validation set?" This is structurally identical to: "among all candidate hyperparameter configurations $\{\lambda_1, \lambda_2, \ldots\}$ of a regularized model, which yields the lowest validation loss?"

6.3 Joint Selection of Model and Threshold

In practice, both the model hyperparameters and the threshold must be chosen from data, which means both choices are subject to the same risk of overfitting to a particular held-out set. The correct framework for handling this jointly is **nested cross-validation**, already covered in the *Model Selection* lecture notes, now applied with threshold selection explicitly in mind.

The two loops serve distinct purposes:

- The **outer loop** provides an unbiased estimate of the generalization error of the fully specified system (model + threshold). Each outer fold yields one held-out test fold that is never touched during any model or threshold decision.
- The **inner loop** runs inside each outer training partition and is responsible for *two* sequential decisions: (i) selecting model hyperparameters, then (ii) selecting the threshold α.

The recommended procedure within each outer fold is:

1. **Outer split**: set aside the outer test fold. It will not be touched until step 4.
2. **Inner CV (model selection)**: run cross-validation on the remaining data to select model hyperparameters (e.g., regularization strength, number of trees). Evaluate each candidate configuration using a *threshold-independent* metric such as PR-AUC or AUC-ROC. This keeps hyperparameter search decoupled from any particular operating point on the score scale.
3. **Threshold selection**: refit the chosen model on the full inner training partition, then sweep α on the inner validation fold to select the value that optimizes the task-specific criterion (F_β, expected cost, or a recall/precision constraint).
4. **Outer evaluation**: apply the model with the fixed $\hat{\alpha}$ to the outer test fold *once* and record the result. Average across outer folds to obtain the final performance estimate.

Why use threshold-independent metrics in the inner loop?

If we optimize F_β at a fixed α inside the inner loop, we are simultaneously searching for the best hyperparameters *and* the best threshold on the same data. These are two separate decisions, and conflating them causes the inner loop to overfit to a particular threshold operating point: the selected hyperparameters may be good only because they happen to pair well with that specific α, not because they produce the best-ranking model in general. Using PR-AUC or AUC-ROC in the inner loop isolates the two decisions cleanly: the inner loop selects the best *ranker*, and the subsequent threshold step finds the best *operating point* for that ranker.

Figure 7 shows one iteration of this nested procedure. The diagram makes explicit a point that is easy to overlook: there are *two* distinct held-out roles in the pipeline, which must not be conflated. The outer test fold is separated at the very beginning and plays no role until step 4; it is the only data against which an unbiased estimate of generalization error can be reported. The training and validation data are used actively throughout the inner loop: first to search for the best-ranking model via PR-AUC, then to select the threshold that best serves the task-specific objective. Only after both decisions have been committed to is the outer test fold evaluated. The key invariant is the same as in standard nested cross-validation: no label from the outer test fold ever feeds back into any modeling decision.

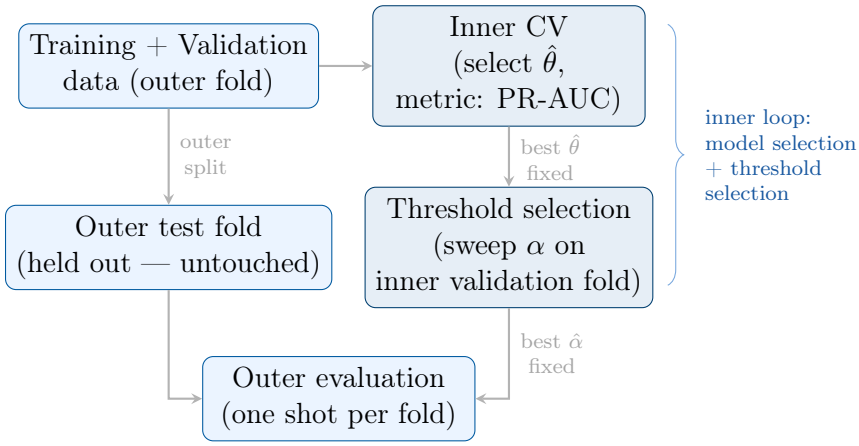

Figure 7: One iteration of nested cross-validation adapted for joint model and threshold selection. The inner loop selects hyperparameters via PR-AUC (threshold-independent), then selects α on the inner validation fold. The outer test fold is evaluated exactly once, after both decisions are fixed. The full procedure repeats for each outer fold; final performance is the average across folds.

The final performance estimate is the aggregate of the per-fold outer results; the two aggregation conventions are discussed in the *Model Selection* lecture notes. For imbalanced problems, the second convention is generally preferred: when the minority class is very rare, some outer folds may contain very few positive examples, making per-fold metrics such as F_1 undefined or highly unstable. Computing the metric once over all concatenated predictions avoids this problem.

Entry Ticket 6

A team trains a fraud classifier using 5-fold cross-validation to select the number of trees in a random forest. They optimize F_1 at $\alpha = 0.5$ inside each fold, then choose the threshold that maximizes F_1 on the test set.

1. Identify two methodological errors in this procedure and relate each one to a specific loop in the nested cross-validation framework.
2. Propose a corrected workflow. Which metric should be used inside the inner loop, and at which step should α be selected?
3. The corrected procedure yields $F_1 = 0.71$ on the test set, while the flawed procedure reported $F_1 = 0.79$. Explain why the flawed estimate is higher, using the concepts of data leakage and overfitting to the threshold.

7 Comparing Strategies: A Practical Overview

No single strategy universally dominates all others. The choice depends on the dataset size, the imbalance ratio, the computational budget, and the model family.

Strategy	Advantages	Disadvantages	Best for
Random under-sampling	Fast; reduces training cost	Discards information; risky for small datasets	Very large datasets
Random over-sampling	Simple; no information loss	Overfitting from duplication; slower training	Small datasets
SMOTE	Generates diverse examples; reduces overfitting vs. random over	Can create noisy examples near boundary; not suitable for all feature types	Tabular data, moderate imbalance
Class-weighted loss	No data modification; easy to tune	May not fully compensate for extreme imbalance	Any model family
Focal loss	Targets hard examples; very effective for deep learning	Additional hyperparameter γ; less intuitive	Deep neural networks
Balanced Bagging / EasyEnsemble	Uses all majority data; robust	Higher training cost; requires ensemble infrastructure	When ensemble is acceptable

Practical recommendation

A reliable starting point for most imbalanced classification problems:

1. Evaluate the baseline model using PR-AUC and F_β (do not rely on accuracy).
2. Apply class-weighted loss (easy, zero extra training cost).
3. Apply SMOTE on the training folds inside cross-validation.
4. Tune the decision threshold α on the validation set using the target metric.
5. Compare strategies using PR-AUC on a held-out test set.

Entry Ticket 7

A team is building a classifier to detect defective products on a factory assembly line. Out of 10,000 products per shift, 50 are defective (0.5 %). Missing a defect costs \$1,000 (recall); a false alarm costs \$20 (one unnecessary inspection).

1. Which metric(s) should the team optimize?
2. Propose a complete pipeline: which data-level and algorithm-level strategies would you combine, and why?
3. How would you choose the final decision threshold? Describe the procedure, specifying which data split (training, validation, or test) is used at each step and why the test set must not be used for threshold selection.

4. At the end, a junior colleague reports "our model achieves 99.5 % accuracy!" How do you respond?

8 Summary

Concept	**Key idea**
Accuracy trap	On imbalanced data, a model that ignores the minority class can achieve very high accuracy; accuracy is not a useful metric in this setting
Default threshold failure	The standard $\alpha = 0.5$ is rarely appropriate when positive examples are rare; models tend to assign low scores to true positives
Majority-class bias	Standard losses are dominated by the majority class; the minority class receives a weak gradient signal during training
PR-AUC vs. AUC-ROC	PR-AUC does not involve TN and is therefore more sensitive to minority-class performance; prefer it for highly imbalanced problems
Threshold selection	Choose α by maximizing F_β, minimizing expected cost, or satisfying operational constraints on recall or precision
Threshold as hyperparameter	α is selected on a validation set, not the test set; joint model and threshold selection requires keeping the two decisions in separate data loops
Random under/oversampling	Simple data-level strategies; undersampling discards majority examples, oversampling duplicates minority examples
SMOTE	Generates synthetic minority examples by interpolating between nearest neighbors; reduces overfitting compared to random oversampling, but may place synthetic points in overlap regions
Class-weighted loss	Assigns higher loss weight to minority-class errors; compensates for imbalance without modifying the data
Focal loss	Dynamically down-weights easy examples via $(1 - \hat{p}_t)^\gamma$; focuses learning on hard examples, which in imbalanced settings are often (but not exclusively) from the minority class
Balanced Bagging / EasyEnsemble	Train each base classifier on a balanced subset; aggregate predictions across the ensemble

Key Takeaways

1. **Accuracy is misleading** on imbalanced datasets; always complement it with recall, precision, F_β, and PR-AUC.
2. **The default threshold $\alpha = 0.5$ is rarely appropriate**; threshold tuning is a necessary final step whose optimal value depends on the cost of FP and FN.
3. **PR-AUC is the primary evaluation metric** for imbalanced problems; AUC-ROC can be overly optimistic because the large TN count suppresses FPR.
4. **Data-level strategies** (undersampling, oversampling, SMOTE) work by balancing the training set; they must be applied inside cross-validation folds to avoid data leakage.
5. **Algorithm-level strategies** (class-weighted loss, focal loss) address imbalance without modifying the data; they are often the simplest first step.
6. **The threshold α is a hyperparameter**: select it on a validation set using the task-specific metric; use threshold-independent metrics (PR-AUC, AUC-ROC) inside the inner model-selection loop to keep the two decisions separate.
7. **No single strategy dominates**; in practice, combining class-weighted loss + SMOTE + threshold tuning is a robust starting point for most problems.

8 References

Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002. doi: 10.1613/jair.953.

Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988, 2017. doi: 10.1109/ICCV.2017.324.

Xu-Ying Liu, Jianxin Wu, and Zhi-Hua Zhou. Exploratory undersampling for class-imbalance learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics*, 39(2):539–550, 2009. doi: 10.1109/TSMCB.2008.2007853.